
Milestone-3 Report: Safety Verification of Chiron IR using Constrained Horn Clauses

Project Overview

Program verification provides mathematical guarantees that software behaves correctly, unlike testing, which can find bugs but never prove their absence. While the Chiron framework offers support for dynamic analysis techniques such as fuzzing, symbolic execution, and spectrum-based fault localization, it currently lacks the capability to *prove* that a program satisfies a given specification for *all* possible inputs.

This project bridges the gap between testing and verification in Chiron by implementing a PDR-based safety verification engine. We target properties natural to turtle graphics programs, such as spatial bounds on the turtle's position and invariants over program variables.

Team Information

Roll No.	Roll No.	Email ID
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in

Implementation

Summary of Implementation Progress

The verifier now supports a complete CHC-based verification flow for Chiron programs: parsing source into IR, extracting user variables and loop counters, constructing symbolic state relations, generating instruction-level transition rules, and checking user properties through Z3 Fixedpoint/SPACER. The core state model includes program counter, turtle coordinates, heading, pen state, user variables, and internal counters, and the tool reports clear verdicts (**PASSED**, **FAILED**, **UNKNOWN**) using a structured return object with timing and per-property outcomes. The verification interface is fully callable through `CHC_Verification(...)` and supports robust input validation, controlled property parsing, and consistent result reporting.

In addition to baseline verification, the implementation includes explicit heading-grid safety checking through a dedicated `BadHeading` relation, configurable scope controls (`all`, `terminating`, `at_pc`, `upto_pc`), and user-selectable hint behavior for performance-vs-assumption tradeoffs. A basic optimization pipeline has been integrated with static turn-safety analysis, repeat-loop summarization, and exact 15-degree trigonometric handling on the heading lattice. To improve usability, a separate semantic helper transpiler was added so readable user-facing helper expressions can be lowered to solver-ready property strings without modifying the core verifier. The implementation is supported by expanded correctness and performance test coverage across modes, scopes, heading behavior, and optimization settings.

Verification Modes and Property Scopes

Verification Modes

1. **default.** Verifies from a single concrete start state: `pc=0`, `xcor=0`, `ycor=0`, `heading=0`, `pendown=False`, all user variables initialized to 0, and all loop counters initialized to 0. This is the baseline configuration for proving invariants from Chiron’s standard start state.
2. **specific.** Verifies from a concrete parameterized start state. The caller supplies values for user variables through `params`; any omitted user variable is defaulted to 0. The rest of the state remains fixed at the standard start configuration (`pc=0`, zero position, zero heading, pen up, counters zero).
3. **universal.** Verifies over a symbolic family of initial states. The verifier fixes `pc=0`, `pendown=True`, and all counters to 0; constrains `heading` to the 15-degree grid; and leaves `xcor`, `ycor`, and user variables unconstrained. Optional `input_ranges` can additionally restrict selected user variables.

Property Scopes

1. **all.** Checks whether the property holds for every reachable state in the verifier’s symbolic model, regardless of control location.
2. **terminating.** Restricts checking to reachable terminal states, i.e. states whose `pc` equals the terminal program counter. If the verifier cannot establish reachability of termination, the result is conservatively reported as `UNKNOWN`.
3. **at_pc.** Restricts checking to reachable states at one exact control point `pc_target`. This scope requires an explicit `pc_target`; if that program counter is unreachable, the scoped query is returned as `UNKNOWN`.
4. **upto_pc.** Restricts checking to the reachable execution prefix satisfying $0 \leq pc \leq pc_target$. This is useful for prefix invariants that should hold before or up to a particular program point.

The API also enforces a few scope-specific restrictions: `pc_target` is required for `at_pc` and `upto_pc`, forbidden for `all` and `terminating`, and the pc-scoped options are disabled when `optimization_level=BASIC` because loop summarization currently targets whole-program or terminal-state reasoning.

Optimization Levels

1. **OptimizationLevel.NONE.** Uses the baseline encoding: the verifier emits the standard per-instruction CHC rules and supports all four property scopes (`all`, `terminating`, `at_pc`, `upto_pc`).
2. **OptimizationLevel.BASIC.** Enables the current optimization pipeline, namely turn-safety suppression of unnecessary `BadHeading` rules, repeat-loop summarization, and conditional branch merging inside summarized loop bodies. This level is intended for whole-program or terminal-state reasoning, so the API rejects `at_pc` and `upto_pc` queries when `BASIC` is selected.

End-to-End Pipeline View

File-wise Implementation Details

1. **CHC-Verification/variable_name_detection_in_IR.py** Extracts user variables and internal repeat counters from linear IR. Also defines optimization-level controls that are consumed during CHC generation. Variable extraction uses recursive AST traversal through `parse_variables_from_ir_expr()` and `parse_variables_from_ir_cond()`, producing a `symbol_table` of user-declared variables and a `counter_table` of loop iteration variables. These two tables determine the arity and argument ordering of the `Inv` relation declared in the next stage, so the accuracy of extraction directly governs the completeness of the symbolic state model.

2. **CHC-Verification/z3_fixed_point.py** Builds the mode-aware state signature and declares core relations such as `Inv` and `BadHeading`. This module defines the relation arity and state-vector shape used by downstream rules. The full state vector carries program counter, turtle coordinates (`xcor`, `ycor`), heading, pen state, all user variables, and all loop counters; both a *current-state* and a *next-state* copy are constructed so transition rules can be expressed as implications between them. `BadHeading` shares the same state-vector shape as `Inv`, allowing heading-violation rules to reuse the same variable symbols and be registered in the same fixedpoint context without duplicating infrastructure.

3. CHC-Verification/init_fixed_point.py Initializes the Z3 Fixedpoint context (SPACER engine), registers relations, and seeds base constraints according to verification mode (`default`, `specific`, `universal`). In `default` mode the initial fact pins all state components to concrete values (`pc=0`, coordinates zero, heading zero, pen up, all variables zero); in `universal` mode coordinates and user variables are left unconstrained while heading is restricted to the 15-degree grid and pen is forced down, capturing the widest possible input class. The `specific` mode accepts a caller-supplied parameter dictionary, initializes the corresponding user variables to those concrete values, defaults omitted user variables to zero, and keeps the remaining state at the standard start configuration. This enables targeted property checking from a concrete parameterized initial state. The same module also normalizes and validates `input_ranges` for universal mode. A variable may be given as a fixed integer, a closed integer interval (`lo`, `hi`), or `None` to leave it unconstrained; unknown variable names, non-integer bounds, and malformed ranges are rejected eagerly before any CHC rules are emitted.

4. CHC-Verification/step_rules.py Translates each IR instruction into CHC transition rules. It covers assignments, conditionals, movement, heading updates, pen-state updates, assertions, and control-flow jumps. This module is also where optimization-aware rule paths and loop summarization hooks are integrated. For movement instructions, the rule encodes exact integer arithmetic for heading multiples of 15 degrees by consulting the precomputed trigonometric table in `optimization_helpers.py`, avoiding floating-point imprecision in the symbolic encoding. In `BASIC` optimization mode, loops that pass the summarizability check are replaced by a single aggregate transition via `summarize_loop_effect()`, reducing the clause count submitted to SPACER and improving convergence on loop-heavy programs while leaving non-summarizable fragments on the standard per-instruction path. Assertions are encoded as hard safety obligations rather than advisory checks: for an `AssertCommand`, the true branch advances normally, while the false branch implies `False` in the fixedpoint system. This turns assertion failure directly into reachability of an inconsistent state, so violated assertions are handled by the same CHC safety machinery as user-supplied properties.

5. CHC-Verification/safety_properties.py Implements the top-level API `CHC_Verification(...)`. It validates inputs, parses hints, applies scope guards, checks heading-grid safety, evaluates user properties, and returns structured outcomes. Each user property is evaluated inside a controlled `eval()` context populated with the current Z3 state variables, so property strings can reference program variables by name without requiring the caller to import Z3 directly. The return object `ReturnValue` bundles an overall verdict (`SUCCESS`, `ERROR`, or `UNKNOWN`), a separate `heading_grid_safe` field, wall-clock build and solve timings, and three property lists (`passing`, `failing`, `unknown`), making outcomes fully machine-readable for downstream test harnesses. This module also contains a small robustness layer around SPACER queries. The helper `safe_fp_query()` retries a query with `spacer.native_mbp=False` when Z3 raises the known `mbp to-real` exception on mixed Int/Real reasoning, and `guard_check()` proactively disables native MBP before reachability checks. This does not change the logical encoding, but it makes the verifier materially more stable on geometry-heavy benchmarks.

6. CHC-Verification/heading_grid.py Defines heading-lattice predicates used to prove or assume 15-degree grid safety. The core function `heading_on_grid()` generates a Z3 disjunction over all 24 valid headings ($0^\circ, 15^\circ, \dots, 345^\circ$), which is embedded directly as a guard in movement and turn transition rules rather than checked separately. This predicate is also the conclusion of the `BadHeading` violation rules: any transition that produces a heading outside the grid implies `BadHeading` on the resulting state, allowing SPACER to derive a reachability witness if the invariant is broken.

7. CHC-Verification/optimization_helpers.py Implements static turn-safety reasoning, repeat-loop detection, summarizability checks, and exact 15-degree trig tables used by optimized rule generation. The precomputed `_MULT15_VALUES` dictionary maps each of the 24 valid headings to exact integer Δx and Δy displacements scaled to avoid fractions, so movement rules submitted to SPACER contain only integer arithmetic and require no real-valued reasoning. The `is_summarizable_loop()` predicate enforces structural constraints: constant loop bound, no nested jumps, no pen-state changes so that the summarization in `step_rules.py` is provably sound: every concrete execution through the loop is covered by the emitted summary transition.

8. CHC-Verification/semantic_properties.py Provides a user-facing helper-string transpilation layer that lowers readable semantic helpers into raw CHC-compatible property expressions without modifying the core verifier. Helper functions such as `is_nonnegative(v)`, `position_in_box(...)`, `pen_is_up()`, and `relation_guarded(...)` are resolved before the property string reaches `eval()`, so the caller can write human-readable specifications that are automatically converted to Z3 Boolean expressions over state variables. The companion entry point `CHC_Verification_semantic()` wraps `CHC_Verification()` with

this transpilation step, keeping the semantic helper layer fully optional and backward-compatible with callers that already supply raw Z3 expressions.

Design Choices

1. **CHCs solved by Z3 SPACER.** The verifier encodes program behavior as Constrained Horn Clauses and submits them to Z3's `Fixedpoint` with `engine='spacer'`, an IC3/PDR-style Horn-clause solver that performs inductive invariant synthesis.
 - *Benefit:* Supports unbounded safety checking; returns a concrete counterexample witness on violation, not just a yes/no answer.
 - *Tradeoff:* Performance is sensitive to nonlinear arithmetic obligations; complex programs or unconstrained inputs can produce UNKNOWN within the timeout.
2. **Single global invariant relation `Inv(pc, xcor, ycor, heading, pendown, vars, counters)`.** Full control-and-data state is captured in one unified relation shared across initialization, all transition rules, and every property query.
 - *Benefit:* `pc` governs control-flow and termination, geometric variables support spatial predicates, and loop counters make repeat semantics explicit without unrolling; all pipeline stages speak the same signature.
 - *Tradeoff:* Properties over a strict variable subset still incur full-arity SPACER reasoning; adding any new state component requires regenerating the entire relation signature and all registered rules.
3. **Strict heading lattice with explicit `BadHeading` relation.** Heading is restricted to $\{0^\circ, 15^\circ, \dots, 345^\circ\}$ and movement arithmetic is handled via a precomputed 24-entry integer lookup table rather than transcendental `sin/cos` constraints.
 - *Benefit:* Keeps all arithmetic in the linear integer arithmetic (LIA) fragment, which SPACER handles reliably; heading violations are a first-class proof obligation rather than a silent assumption.
 - *Tradeoff:* The disjunctive heading guard expands CHC clause size proportionally to the number of valid headings; failing to prove grid preservation yields a conservative UNKNOWN on all downstream property checks.

Optimisations

Files: `CHC-Verification/optimization_helpers.py`, `CHC-Verification/step_rules.py`

The following optimisations are implemented at `OptimizationLevel.BASIC` to reduce the number and complexity of CHC rules submitted to SPACER, improving performance on loop-heavy programs and those with many turn instructions while preserving soundness.

Heading Check Optimisation

Each turn instruction (`left/right`) in the standard encoding generates a `BadHeading` CHC rule asserting that the resulting heading might leave the grid. The heading-check optimisation eliminates these rules wherever the turn amount is statically proven to be a multiple of 15.

1. A forward data-flow analysis (`compute_env_in`) propagates, for each program point, the set of variables known to hold a multiple-of-15 value. The transfer function `transfer_env` updates this set on assignments: if the right-hand side evaluates to `True` under `is_multiple_of_15`, the left-hand variable is added; otherwise it is removed.
2. At merge points (after conditionals), `join_envs` takes the intersection a variable is kept only if it is provably a multiple of 15 on *every* incoming path, preserving soundness across branches.
3. `is_multiple_of_15` handles numeric literals (`n % 15 == 0`), variable lookups, unary minus, sums and differences (both operands must be known), and multiplication (either factor must be known). Any unsupported shape returns `None` (unknown), triggering conservative `BadHeading` emission.
4. If a turn's delta resolves to `True` at its program point, `turn_safe_map[i]` is set and the `BadHeading` rule for that instruction is suppressed, reducing the clause count submitted to SPACER.

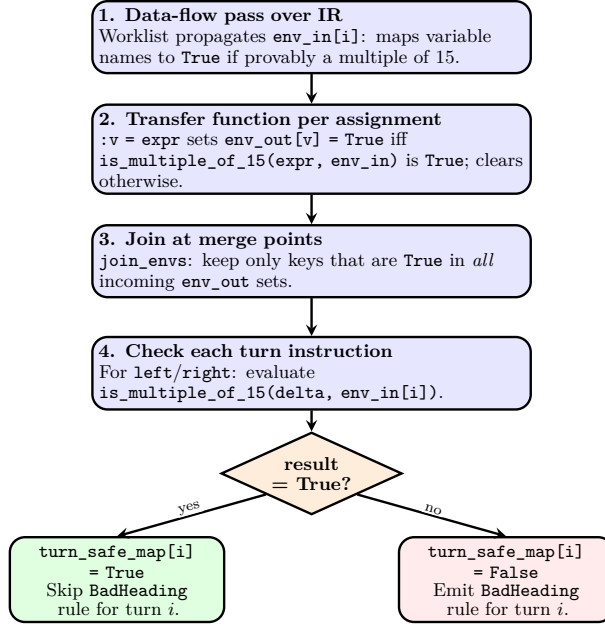


Figure 1: Turn-safe heading inference: data-flow analysis that marks turn instructions whose delta is statically provable as a multiple of 15.

Loop Optimisation

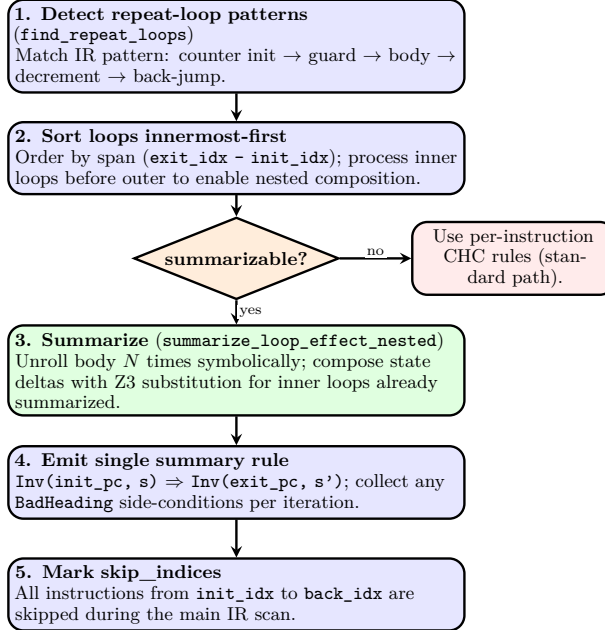


Figure 2: Repeat-loop summarization: innermost-first pattern matching, summarizability check, symbolic unrolling, and single-rule emission.

Instead of emitting one CHC rule per instruction for every iteration of a **repeat** loop, the loop optimisation compiles the entire loop into a single transition from loop entry to loop exit.

1. `find_repeat_loops` scans the IR for the canonical 6-instruction repeat pattern: counter initialisation, a `counter > 0` guard, a loop body, a `counter = counter - 1` decrement, and a back-jump. This pattern is produced deterministically by Chiron’s parser.

2. `is_summarizable_loop_nested` checks structural preconditions: the loop count must be a positive integer not exceeding `MAX_SUMMARIZE_ITERATIONS` (150), the body must contain no `AssertCommand` nodes, and every non-conditional instruction must have unit jump offset. Already-summarized inner loops are treated as opaque atomic blocks during this check.
3. Loops are processed innermost-first. `summarize_loop_effect_nested` unrolls the body N times symbolically, carrying a running Z3 state expression rather than adding solver variables. When an inner loop is encountered, its pre-computed state transformation is applied via Z3 `substitute` rather than stepping through its instructions again.
4. The result is one `fp.rule(ForAll(vars, Inv(init_pc, s)  $\Rightarrow$  Inv(exit_pc, s')))` rule replacing the entire loop, plus any `BadHeading` side-condition rules accumulated during unrolling. All instructions within the summarized range are added to `skip_indices` and bypassed in the main IR scan.

Conditional Optimisation

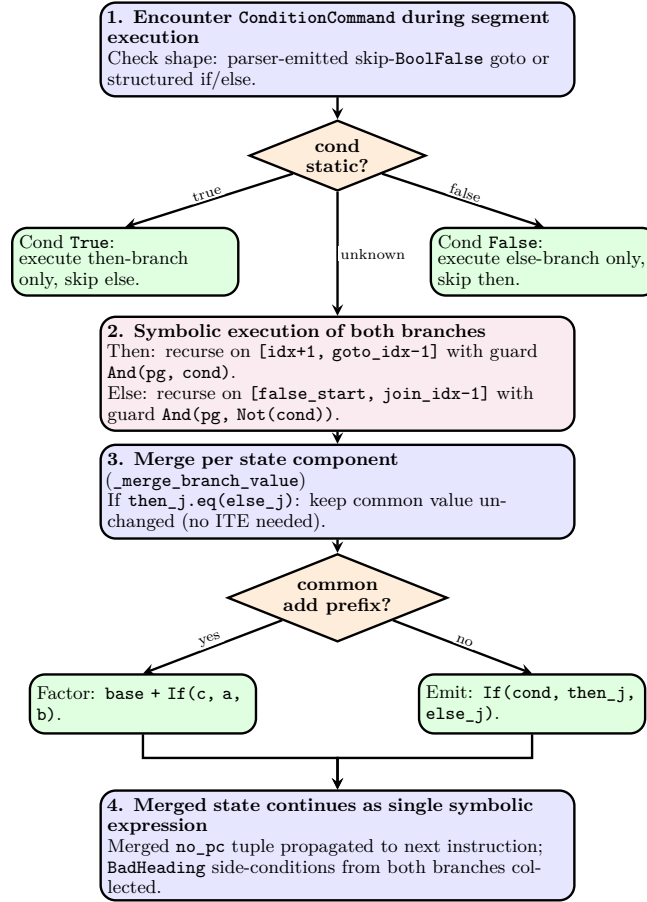


Figure 3: Conditional handling inside summarized loop bodies: static branch elimination and ITE merging with common-prefix factoring.

When a loop body contains if/else branches, the summarizer must represent both paths as a single symbolic state expression rather than splitting into multiple CHC rules.

1. `_execute_segment_with_branch_merge` handles two conditional shapes produced by Chiron's parser: a `BoolFalse` skip-goto (used to jump over the else block at the end of the then-branch), and a full structured if/else with a true-branch terminator goto followed by the else block.
2. If the branch condition is statically `True` or `False` at summarization time (Z3's `is_true/is_false`), the dead branch is dropped entirely and no `If`-expression is introduced, keeping the resulting CHC rule compact.

3. For the general case, both branches are executed symbolically under their respective path guards (`And(pg, cond)` and `And(pg, Not(cond))`), and each state component is merged independently using `_merge_branch_value`.
4. `_merge_branch_value` first checks if then and else values are syntactically equal (`then_j.eq(else_j)`); if so, the common value is kept unchanged without any `If`. Otherwise it looks for a common additive prefix: `If(c, base+a, base+b)` is rewritten to `base + If(c, a, b)`, reducing expression depth in branch-heavy loops. If neither shortcut applies, a plain `If(cond, then_j, else_j)` is emitted.
5. `BadHeading` side-conditions accumulated in both branches are collected and emitted together with the summary rule, so heading-grid obligations inside conditional bodies are not lost.

Hints

File: `CHC-Verification/safety_properties.py`

Hints are optional caller-supplied strings passed via the `hints` parameter that let the user trade a proof obligation for an assumption, skipping a solver query when the property is already known to hold. All hints are validated at API entry against the `Hints` enum; an unrecognised value is rejected before any solving begins.

- **check_heading_always_on_grid** (default). Proves that the turtle’s heading stays on the 15-degree grid by querying `BadHeading`. The result is stored in `ReturnValue.heading_grid_safe`; if the check fails or is unknown, the verifier stops before evaluating user properties.
- **heading_on_grid_always**. Assumes the heading invariant rather than proving it. The `BadHeading` query is skipped and `heading_on_grid(state[3])` is added directly as an assumption. Useful when all turns are provably multiples of 15 by construction, saving one SPACER call.
- **check_termination** (default). When `property_scope="terminating"`, first queries whether the terminal PC is reachable before checking properties there. If SPACER cannot confirm termination, the run returns UNKNOWN without attempting the property checks.
- **always_terminates**. Skips the termination reachability check and assumes the terminal state is reachable, adding the terminal-PC guard directly as an assumption. Appropriate for programs with only finite repeat loops and no infinite branches.

The two heading hints and the two termination hints form mutually exclusive pairs: supplying both members of a pair is rejected at validation since one proves and the other assumes the same property.

Future Work

Future work to make the verifier stronger and easier to use includes:

1. **ML-guided verification.** Many results become UNKNOWN because hints and solver settings are fixed. A useful next step is to use lightweight ML models over IR features (loop depth, branching, variable count, and property form) to choose hints, timeout values, and optimization levels automatically. This can reduce failed runs and improve solver performance on different program families.
2. **Generalize beyond the current heading-grid discipline.** The current verifier assumes heading values stay on a 15-degree grid. If this cannot be proved, verification often returns UNKNOWN. We plan to extend support to arbitrary angles by adding stronger handling of non-linear trigonometric updates, using sound abstractions, piecewise-linear approximations, or non-linear solver backends.
3. **A portfolio of multiple SMT solvers.** The current pipeline depends on one Horn-clause solving stack. A solver portfolio can run multiple backends with fallback per property and return the strongest result available (PASSED/FAILED preferred over UNKNOWN) under a shared timeout policy.
4. **Extra properties and richer user semantics.** Basic arithmetic, geometry, pen, and heading helpers already exist. Next, we should add higher-level property types such as temporal properties (`always(phi)`, `eventually(phi)`, `until(phi, psi)`), bounded-step goals (`within_k_steps(phi, k)`), monotonicity checks (`monotonic_nondecreasing(v)`), and richer geometric regions (such as polygon containment). This would let users express trace-level behavior, not just state-level constraints, while still compiling safely to CHC formulas.
5. **Counterexamples that teach.** Failed queries currently return solver-level outputs that are hard to read. Future versions should rebuild source-level traces, mark the first instruction where the property breaks, and show a minimal witness over only relevant variables. This would make failures easier to debug and learn from.

6. **Scalability.** Universal mode and deep control flow increase relation size and rule count, which hurts runtime and raises UNKNOWN rates. A key direction is parallelism: multi-threaded checking of independent property batches, concurrent solver portfolios with shared timeout budgets, and parallel preprocessing (IR analysis, slicing, and summarization). Along with compositional summaries and incremental caching, this can improve throughput and reduce wall-clock time on multi-core machines.
7. **Optimisations.** Beyond current basic optimizations, we plan the following sound performance improvements:
 - **Property-impact slicing:** keep only instructions and variables that can affect a given property, and generate smaller CHC systems.
 - **Adaptive timeout scheduling:** assign per-property timeout budgets from simple complexity signals instead of one fixed timeout.
 - **Parallel property queues:** solve independent properties on worker threads and merge results in deterministic API order.
 - **Incremental lemma reuse:** reuse learned invariants across scopes (`all`, `terminating`, `at_pc`, `upto_pc`) for the same IR.
 - **Counterexample-guided refinement:** when a query is UNKNOWN, refine only ambiguous transitions instead of strengthening the full model.

Source Code

The complete implementation is hosted at: <https://github.com/chiron-verification/milestone-2>

- **Milestone-1 commit:** 0f99ade6d4aa2516f9e47e013ad06d29e8b8a2f7
- **Milestone-2 commit:** 4d3e3e6499b5eea03d25f0b0a3e1bf7a4dcd998b
- **Milestone-3 commit:**

The correctness test suite (`correctness_tests/`) has substantially expanded across `test_default.py`, `test_specific.py`, `test_universal.py`, and `test_semantics.py` along with many new `.tl` fixtures and shared helpers to improve coverage of arithmetic, control-flow, and heading-related properties.

In addition, a dedicated performance testing setup (`performance_tests/`) is added with mode-wise test runners, benchmark fixtures, and reusable harness/configuration files to measure solver behavior on deep nesting, wide-state, branch-heavy, and high-iteration workloads.

AI Use in the Project

In this project, we utilized AI tools to enhance our workflow and improve the quality of our work.

1. **Are you using AI for coding your implementation?** Yes, we used AI tools in this project.
2. **If yes, what models are you using?** We used models like OpenAI's GPT-5.3 Codex, GPT-5.4 Codex, Claude Code and GitHub Copilot.
3. **If using AI, how are you using AI and what percentage (approx) of your code is AI generated?** We used AI primarily for generating boilerplate code, suggesting improvements, and debugging. Approximately 40% – 50% of our code was generated or assisted by AI tools.
 - AI was primarily used for generating boilerplate code and usage of correct syntax, which was then reviewed and modified by us to fit our specific requirements.
 - We also used AI for testing, where it helped in generating test cases and suggesting improvements to our existing tests.
 - Additionally, AI was used for the generation of documentation and figures in the report, where it assisted in summarizing the directory structure and usage guide, as well as making the figures in the report with careful and structured instructions about the layout.

Declaration

1. We declare that the work presented in this report is our own and has been carried out in accordance with the guidelines set out by the course.
2. We have not copied any part of this report from any other source, and we have not submitted this report to any other course or institution for assessment.
3. We understand that any violation of this declaration may result in disciplinary action.